Reproducing AMR-EnsembleNet: A 1D CNN–XGBoost Ensemble for Antimicrobial Resistance Prediction

Milestone 3 Report

Muhammad Zaid Bilal (ERP 30506) Omer Ibrahim Qazi (ERP 30517) Haseeb Adnan (ERP 30483)

Institute of Business Administration (IBA), Karachi
Introduction to Machine Learning — Instructor: Dr. Rizwan Ahmed Khan
October 20, 2026

This report reproduces and experiments on Siddiqui and Tarannum (2026), "Fusing Sequence Motifs and Pan-Genomic Features: Antimicrobial Resistance Prediction using an Explainable Lightweight 1D CNN–XGBoost Ensemble," published at SCA/HPCAsia 2026. Our code is available at `https://github.com/oliGAMER/ML_Paper_Reprod`*, branch* `M3_Own_Experiment`*.*

1 Introduction and Problem Statement

Antimicrobial resistance (AMR) occurs when bacteria survive the drugs meant to kill them. It is a growing public-health problem. The paper cites a 2016 review and WHO estimates that drug-resistant infections could cause around 10 million deaths a year by 2050. A major reason is slow diagnosis: standard culture tests take two to three days to show which antibiotic will work, so clinicians often prescribe broad-spectrum drugs, which can encourage further resistance.

Whole-genome sequencing (WGS) offers a faster route. In principle, a bacterium's DNA reveals its resistance profile within hours. Resistance arises from single nucleotide polymorphisms (SNPs), which are single-base differences between isolates, and from acquired resistance genes. Once a genome is sequenced, each isolate can be represented as a row of SNP values, and predicting resistance becomes a supervised classification problem: given the SNPs, predict resistant or susceptible.

Two families of models are common for this task, and each has a gap. Standard machine learning models such as Random Forest and XGBoost treat SNPs as an unordered bag of features and ignore their position in the genome. This loses information when several nearby mutations act together within one gene. Sequence models such as Transformers can use order, but they usually need much more data than the moderately sized genomic datasets available.

The problem this project addresses is whether a lightweight model that reads SNPs in order, combined with a tree-based model that captures interactions regardless of position, predicts resistance more reliably than either model alone. The paper answers yes with AMR-EnsembleNet, a soft-voting ensemble of a 1D convolutional neural network (CNN) and XGBoost. It is evaluated on 809 *E. coli* isolates across four antibiotics: ciprofloxacin (CIP), cefotaxime (CTX), ceftazidime (CTZ) and gentamicin (GEN).

Our work reproduces the paper's method, compares our results with its reported numbers, and runs an additional experiment on how the ensemble weighting and the number of SNP features affect performance.

2 Literature and Background

Biological background. Fluoroquinolones such as ciprofloxacin act on two bacterial enzymes, DNA gyrase and topoisomerase IV, which are needed for DNA replication. Resistance often comes from point mutations in the genes that encode these enzymes (*gyrA* and *parC*, and to a lesser extent *gyrB*). Eaves et al. (2004) and Hooper (2001) describe the quinolone-resistance-determining regions where these mutations cluster. Efflux pumps and beta-lactamases are other common routes, which matters for cefotaxime and ceftazidime.

Machine learning on bacterial genomes. Before deep learning, most work used SNP matrices with tree ensembles. Drouin et al. (2016) applied random forests to reference-free genome comparisons for phenotype prediction. Gygli et al. (2019) showed that whole-genome sequencing plus machine learning can predict tuberculosis drug resistance. Li et al. (2018) selected SNP subsets with several machine-learning methods, and Shi et al. (2019) used deep feature selection to find resistance factors. These methods treat each SNP as an independent column, so they cannot use the order of nearby mutations. Sakagianni et al. (2023) reviewed this area.

The GieSSen dataset. Ren et al. (2022) built the benchmark used here: 809 *E. coli* isolates with SNP profiles and susceptibility labels for four antibiotics. They also benchmarked standard models on it, which gives the paper a baseline.

Sequence models for genomes. Two families of deep models read sequences directly. Transformers such as BERT (Devlin et al., 2019) and DNABERT (Ji et al., 2021) reach strong results, but they are pretrained on very large corpora. A survey by Zhang et al. (2023) notes that training one from scratch on moderately sized genomic data risks overfitting. Convolutional networks are a lighter alternative: van den Belt et al. (2024) used CNNs on allele-frequency encodings, and Sharma et al. (2025) compared encodings for 1D CNNs on genomic sequences.

Where this paper sits. The CNN and XGBoost components are standard. The contribution the paper claims is combining them: the CNN captures local, ordered SNP patterns, and XGBoost captures interactions across the genome. It evaluates that combination on the GieSSen benchmark and compares it with the individual models. Our project checks those claims and tests two design choices the paper fixes: the 50/50 weighting and the use of all 60,936 features.

Citing works. As of October 2026, we found no published work citing AMR-EnsembleNet — an OpenAlex query on its DOI returns zero citing works, and a web search surfaces only the paper's own preprint/ACM listing and our course fork. This is not surprising for a paper published in January 2026, shortly before this report was written.

3 Paper Methodology

The paper proposes AMR-EnsembleNet, which predicts antibiotic resistance from SNP profiles by combining two models. Each model sees the same input, a row of 60,936 integer-coded SNP values per isolate, and each produces a probability that the isolate is resistant. Their outputs are then averaged.

Input. Each SNP value is an integer from 0 to 4, one per nucleotide state. The paper treats the columns in chromosomal order, so the sequence of values reflects the order of the mutations in the genome.

Model 1: 1D convolutional neural network. The network reads the SNP sequence like a signal and is built from four stages:

1. **Embedding:** each integer token is mapped to a learned 64-dimensional vector. The paper argues that a 5-token vocabulary does not need a larger space, and that a larger one would add parameters and overfitting risk.
2. **Six convolutional blocks.** Each block applies a set of learned filters that scan along the sequence, then batch normalisation and a ReLU activation. Early blocks use wide filters (128 filters of width 7) to pick up broader patterns, and later blocks use narrower filters (64 of width 5, then 32 of width 3) for finer detail. Max pooling halves the sequence after the first four blocks, which widens the view of later layers.
3. **Global max pooling.** For each filter, keep only its strongest response anywhere along the genome. The authors argue that resistance is often driven by a few sparse, high-impact changes, so the strongest signal matters more than the average.
4. **Classification head.** A dense layer of 128 units, dropout, and a single sigmoid output that gives the probability of resistance. The paper also mentions L2 regularisation.

Model 2: XGBoost. A gradient-boosted tree model (XGBoost; Chen and Guestrin, 2016) with 1000 trees, maximum depth 6, learning rate 0.05, and 70% of features and 70% of isolates sampled for each tree. It works on the same SNP matrix but ignores the order of the columns.

Baselines. A random forest (Breiman, 2001) of 400 trees, maximum depth 15, with $\sqrt{\text{features}}$ considered at each split, and standalone versions of the CNN and XGBoost, so the ensemble can be compared with each part.

The ensemble. AMR-EnsembleNet is a soft-voting ensemble: the CNN's probability and XGBoost's probability are averaged with equal weight (50/50), and the result is turned into a resistant or susceptible label using a decision threshold. The paper does not say how that threshold was chosen.

Class imbalance. For CTX, CTZ and GEN, the training loss weights resistant isolates more heavily. The paper describes the weights as inversely proportional to the square root of class frequency. This differs from the code, which uses inverse frequency, and we discuss that in Section 4.

Evaluation. Each isolate set is split 80/20 into training and test sets, stratified so both keep the same class proportions. The paper reports accuracy, MCC, Cohen's kappa, macro F1, and precision, recall and F1 for the resistant class.

4 Implementation Details

This section describes what we built and ran. Where we departed from the paper or the original code, we say so and point to Section 8 for the effects.

Environment. We trained on Google Colab with a free NVIDIA T4 GPU, using Python 3.11 and TensorFlow 2.20.0. The paper's requirements file pins TensorFlow 2.18.0, but that version is not installable on Colab's current platform, so we used the nearest available version. XGBoost is pinned to 2.0.3, as in the paper. The random seed is fixed at 42 for the data split, NumPy and TensorFlow.

Data. Each row is one isolate and each column is one SNP position. The 60,936 SNP values are used as integer tokens 0–4 without further transformation. For each antibiotic, the 809 isolates are split 80/20 into training and test sets. The split is stratified, so both sets keep the same proportion of resistant isolates, and it uses seed 42.

CNN. The architecture follows the paper's description (checked against its Figure 1). It has a 64-dimensional embedding, then six convolutional blocks with 128, 128, 64, 64, 32 and 32 filters and kernel widths 7, 7, 5, 5, 3 and 3. Each convolution is followed by batch normalisation and ReLU. Max pooling follows the first four blocks, and dropout of 0.15 follows the last two. Global max pooling is followed by a dense layer of 128 units with dropout of 0.45, and a single sigmoid output gives the probability of resistance. L2 regularisation of 0.001 is applied to the convolution and dense weights.

CNN training. We use AdamW with a learning rate of 1×10^{-3} and a batch size of 32, for up to 150 epochs. Training stops early if the monitored metric stops improving. The learning rate is cut by a factor of 5 after four epochs without improvement in validation loss. The monitored metric and patience differ by antibiotic, as set in the original notebooks: AUC with patience 60 for CIP and GEN, accuracy with patience 40 for CTX and CTZ.

XGBoost. 1000 trees, maximum depth 6, learning rate 0.05, and 70% of isolates and 70% of features sampled per tree. The `scale_pos_weight` parameter is set to the number of susceptible training isolates divided by the number of resistant ones, which balances the classes in the loss. Early stopping uses a patience of 50 rounds.

Decision thresholds. A decision threshold turns a probability into a label: resistant if the probability is at or above the threshold. The paper does not say how its threshold was chosen. We use the per-antibiotic values from the original single-model notebooks: 0.30 (CIP), 0.50 (CTX), 0.62 (CTZ) and 0.505 (GEN). The original ensemble notebook uses different values, 0.55 for CIP and CTZ and 0.48 for CTX and GEN. Those values are hard-coded, and the notebook applies them to the test set. Where the single-model values came from is undocumented.

Class weights. The training loss weights each class by $N/(2n_k)$, where N is the number of training isolates and n_k the number in class k. This makes the minority class count more. The paper describes weights proportional to the inverse square root of class frequency. We kept the code's formula because the team had not yet decided which to report.

Deviations from the paper and original code. Each row below affects the reported numbers. Section 8 discusses their effects.

Item	Paper / original code	Our implementation
Validation data	Split carved from training	Test set used for early stopping, checkpointing and learning-rate reduction
GEN early-stopping monitor	`val_accuracy`	`val_auc`
Class-weight formula	Inverse square root of frequency	Inverse frequency (code's formula)
Decision thresholds	Not stated in the paper	Per-antibiotic values from the original notebooks, applied to test data
TensorFlow version	2.18.0	2.20.0
CTX checkpoint filename	Original notebook overwrote CTZ's file	Filename derived from the antibiotic name

Code provenance. We label each part of the code by where it came from. The CNN architecture and its hyperparameters are adapted from the authors' four per-antibiotic notebooks: we kept their design and settings, but merged the four notebooks into one training script and changed the data flow. The XGBoost

training code is also adapted from the authors' XGBoost notebook. The data loader and the weight-sweep and k-sweep notebooks are our own work.

5 Dataset and Preprocessing

Source. The data is the GieSSen benchmark for *E. coli* antimicrobial resistance, curated by Ren et al. (2022) and distributed through the ML-iAMR repository. The paper uses two files: a SNP matrix and a phenotype file. We use the same two files, which are included in our fork.

Isolates and labels. The dataset contains 809 *E. coli* isolates. Each isolate is one bacterial sample. Each has a phenotype label for each antibiotic: 0 for susceptible or 1 for resistant. The labels come from laboratory susceptibility tests, not from the genome sequence. The four antibiotics are ciprofloxacin (CIP), cefotaxime (CTX), ceftazidime (CTZ) and gentamicin (GEN). Each antibiotic is a separate binary classification task that uses the same SNP features.

Class balance. We computed these counts from the phenotype file, and they match the paper's Table 1.

Antibiotic	Susceptible (0)	Resistant (1)	Resistant share
CIP	443	366	45.2%
CTX	451	358	44.3%
CTZ	533	276	34.1%
GEN	621	188	23.2%

GEN is the most imbalanced: about one isolate in four is resistant. A model that always predicts "susceptible" would be correct 77% of the time on GEN, which is why we report MCC and macro F1 alongside accuracy.

SNP matrix. The matrix has 809 rows (isolates) and 60,936 SNP columns, plus an identifier column. Each column is a site that varies across the collection, in chromosomal order, measured against the *E. coli* K-12 MG1655 reference. Each cell holds the nucleotide at that site for that isolate, stored as an integer: N (no variation) = 0, A = 1, G = 2, C = 3, T = 4. This mapping comes directly from ML-iAMR's own preprocessing script (`02_preprocessing.R`), and our CNN training code uses the same mapping. The k-sweep notebook's SNP-logo visualisation uses a different, incorrect mapping (C and G swapped); this affects only the axis labels on that figure, not any reported metric. The matrix contains no empty cells.

Column names. The 60,936 columns correspond to 16,446 distinct base positions. Many names carry suffixes such as `.1`, `.2` or `.3`. We traced this to ML-iAMR's preprocessing script, which builds and filters a separate SNP matrix for each antibiotic — keeping a position only if it is present in more than 20% of that antibiotic's samples — before the four matrices are combined into one file. Checking this against the data confirms it: 14,683 of the 16,446 base positions (89%) appear exactly four times, once per antibiotic, and the remainder appear once to three times, which is what independent per-antibiotic filtering would produce. We treat each column as a separate feature, as the original pipeline does.

Preprocessing. We apply no transformation beyond the integer encoding. For each antibiotic, the isolates are split 80/20 into training (647) and test (162) sets, stratified by label so both sets keep the same proportion of resistant isolates. The split uses seed 42.

6 Reproduction Results

This section reports the metrics from our reproduction next to the paper's values. The team retrained every model on Colab. We did not use the authors' saved weights. All numbers are on the 162-isolate test split. As Section 4 explains, the test set also guided early stopping and the decision thresholds, so our numbers are probably optimistic. Section 8 discusses how much this matters.

Single-model results. Each model was trained once with seed 42, except where noted. ΔMCC is our MCC minus the paper's.

1D CNN

Antibiotic	Source	Accuracy	MCC	Macro F1	Resistant recall	ΔMCC
CIP	Paper	0.9568	0.9129	0.9564	0.9589	
	Ours (re-run)	0.9198	0.8405	0.9194	0.9452	−0.072
CTX	Paper	0.7840	0.5647	0.7821	0.7778	
	Ours	0.8025	0.6030	0.8010	0.8056	+0.038
CTZ	Paper	0.8086	0.5651	0.7816	0.6727	
	Ours	0.7963	0.5402	0.7698	0.6727	−0.025
GEN	Paper	0.7346	0.3984	0.6836	0.7105	
	Ours (`val_auc` monitor)	0.7778	0.3136	0.6495	0.3684	−0.085

Two runs need explaining:

- **CIP:** the first run gave MCC 0.7568 and recall 0.9178. We report the re-run. Both runs use the same settings, so the gap reflects run-to-run variation, and one seed per run cannot show how large that variation is.
- **GEN:** with the paper's `val_accuracy` monitor, the model predicted every test isolate as susceptible (MCC 0.0, recall 0.0). Switching the monitor to `val_auc` gave the result above. The paper's recall of 0.71 is not reproduced.

XGBoost

Antibiotic	Source	Accuracy	MCC	Macro F1	Resistant recall	ΔMCC
CIP	Paper	0.9630	0.9253	0.9625	0.9452	
	Ours	0.9630	0.9252	0.9626	0.9589	−0.000
CTX	Paper	0.8025	0.6050	0.8014	0.8194	
	Ours	0.7963	0.5937	0.7954	0.8194	−0.011
CTZ	Paper	0.7901	0.5363	0.7680	0.7091	
	Ours	0.7778	0.5139	0.7564	0.7091	−0.022
GEN	Paper	0.7593	0.3722	0.6841	0.5789	
	Ours	0.7716	0.3930	0.6955	0.5789	+0.021

The paper's XGBoost CTX row is identical to its Random Forest row, which may be a copying error in the paper. We report it as published.

Ensemble. Here we add the 50/50 soft-voting ensemble using the paper's own fixed decision thresholds (0.55 for CIP and CTZ, 0.48 for CTX and GEN), so the comparison sits on equal footing with the paper's

reported ensemble numbers. We retrained the CNN and XGBoost ourselves and combined their test-set probabilities with equal weight; we did not use the authors' saved weights.

Antibiotic	Threshold	Paper's ensemble MCC	Our ensemble MCC	ΔMCC
CIP	0.55	0.9260	0.9003	−0.026
CTX	0.48	0.6241	0.5937	−0.030
CTZ	0.55	0.5715	0.5204	−0.051
GEN	0.48	0.4030	0.3228	−0.080

Our ensemble is below the paper's on all four antibiotics, by a larger margin than either single model shown above. Section 8 discusses why: in short, the CNN component we reproduced is the weaker of the two to begin with, which pulls an equal-weight blend down more than it would if the CNN matched the paper's reported performance.

Summary of what we see here, without interpretation. Our XGBoost results are within about 0.02 MCC of the paper's on every antibiotic. Our CNN results are less stable: CTX is above the paper, CTZ is close, and CIP and GEN are below it, with GEN recall well short of the paper's.

7 Additional Experiment

The proposal committed to two Stage 4 experiment types. Both are run here, and the k-sweep's work lives on the `M3_Own_Experiment` branch, as the assignment asks.

7.1 What the additional experiments were testing, and why they are not in the paper

Neither the weight sweep nor the k-sweep has an equivalent in the original paper. The paper fixes the ensemble blend at 50/50 and the feature set at all 60,936 SNP columns without testing either choice. Our experiments exist specifically to question those two fixed choices, which is why they count as additional experiments rather than a second reproduction of something the authors already did.

7.2 Ensemble weight sweep (hyperparameter study)

Question: does the best CNN/XGBoost blend weight shift as the resistance task becomes more imbalanced, and can a tuned weight beat the paper's fixed 50/50 split?

Method. Using the saved CNN and XGBoost models and the same test split as the paper, we swept the blend weight from 0.0 (pure XGBoost) to 1.0 (pure CNN) in steps of 0.05, for each antibiotic. At each weight we computed AUC, which does not need a threshold, and also picked a threshold on the test ROC curve (Youden's J statistic) to report MCC. As Section 4 discloses, picking the threshold this way makes every point in the sweep optimistic, including the 50/50 point, so the 50/50 numbers here are not the same as the paper's 50/50 result at its fixed threshold (Section 6).

Results, by AUC (threshold-free, the more trustworthy signal):

Antibiotic	50/50 AUC	Best AUC	Best weight	Gain
CIP	0.9795	0.9849	0.00 (pure XGBoost)	+0.0054
CTX	0.8839	0.8889	0.00 (pure XGBoost)	+0.0049
CTZ	0.8169	0.8340	0.00 (pure XGBoost)	+0.0171
GEN	0.7570	0.7627	0.10 (XGBoost-leaning)	+0.0057

Results, by MCC (threshold chosen per weight on the test set, less trustworthy):

Antibiotic	50/50 MCC	Best MCC	Best weight
CIP	0.9008	0.9252	0.00 (pure XGBoost)
CTX	0.6706	0.6750	0.65 (CNN-leaning)
CTZ	0.5806	0.5892	0.25 (XGBoost-leaning)
GEN	0.4063	0.4063	0.45 ($\approx$ 50/50)

Reading these results. On the cleaner, threshold-free AUC measure, pure or near-pure XGBoost beats the 50/50 blend on all four antibiotics, though the gains are small (0.005–0.017). The MCC numbers broadly agree for CIP and CTZ, but disagree with AUC on CTX's preferred direction (CNN-leaning vs. XGBoost-leaning), which is a sign that the per-weight threshold choice is adding noise on top of the real signal. So the honest answer to the question is: the paper's fixed 50/50 split is not weight-optimal, XGBoost deserves more weight than the paper gives it almost everywhere, but the exact best weight shown here should not be read as precise.

7.3 Feature-selection k-sweep (feature-scaling study)

Question: does XGBoost need all 60,936 SNP features, or does a smaller, statistically chosen subset perform as well?

Method. We scored every feature with a custom chi-squared test, computed on the training split only so the selection step does not see test labels. We tried k values log-spaced from 100 to 60,936, plus each antibiotic's Bonferroni-significant feature count, and retrained XGBoost from scratch at each k. This XGBoost uses different settings from the main reproduction (300 trees instead of 1000, no early stopping), so its numbers are not directly comparable to Section 6's XGBoost results.

Antibiotic	Best k	Best MCC	Full-feature MCC (60,936)
CIP	$\sim$3,500–3,900	0.9008	0.8654
CTX	60,936 (full set)	0.6278	0.6278
CTZ	$\sim$415–847	0.5960	0.5806
GEN	415	0.4403	0.3249

A caution on the size of these differences. We also ran $k = 60{,}935$, one feature short of the full set, as a byproduct of how the k values were generated. CTX's MCC moved from 0.5937 (60,935) to 0.6278 (60,936), a swing of 0.034 from dropping a single feature. That is larger than several of the "effects" in the table above. With only 162 test isolates and k chosen from 12 candidate values by looking at the test score, some part of every number in this table is noise rather than signal.

Reading these results, with that caution in mind. CIP, CTZ and GEN all reach their best or near-best score with a few hundred to a few thousand features, well under 10% of the full set. CTX is the one antibiotic where the full set is clearly best, and even that conclusion should be treated carefully given the swing above. GEN's result is the most interesting: feature selection outperforms the main reproduction's full-feature XGBoost (0.3930) using a much smaller, differently tuned model. That is worth following up with a validation-set-selected k before it goes in the report as a confident finding.

Both experiments point the same direction: the paper's two fixed choices, the 50/50 blend and the full feature set, are not optimal for most antibiotics, though neither experiment currently isolates the size of the effect from test-set selection noise.

8 Analysis and Discussion

8.1 What the additional experiments were testing, and why they are not in the paper

Neither the weight sweep nor the k-sweep has an equivalent in the original paper. The paper fixes the ensemble blend at 50/50 and the feature set at all 60,936 SNP columns without testing either choice. Our experiments exist specifically to question those two fixed choices, which is why they count as additional experiments rather than a second reproduction of something the authors already did.

8.2 The test-set leakage problem compounds differently in each experiment

Section 4 already disclosed that the paper's own ensemble thresholds (0.55 / 0.48) look like they were chosen by checking test-set performance once. Our weight sweep has a worse version of the same problem, in two layers: for every one of the 21 weights we tried, we pick a threshold by looking at the test ROC curve (layer 1), and then we report whichever weight's result looks best (layer 2). So the headline "best weight" number has been optimised against the test set twice over, not once. The k-sweep only has one layer of this problem — the chi-squared feature selection itself is computed on the training split only, so that step is clean, but the choice of which k value to report as "best" is still made by looking at test MCC. That makes the k-sweep slightly more defensible than the weight sweep, even though both ultimately picked their headline number using the test set.

The practical effect: none of the "best weight" or "best k" numbers in Section 7 should be read as a claim that this exact weight or exact k would generalise to new data. They are evidence about direction (XGBoost deserves more weight than 50/50 almost everywhere; a few hundred to a few thousand SNPs often beat the full set), not about the precise optimum.

8.3 Why CIP so strongly favours pure XGBoost

The weight sweep's clearest result is CIP: the best weight is 0.00, pure XGBoost, with MCC 0.9252, against pure CNN's MCC of 0.8522 at weight 1.00. That is a large gap for a single antibiotic. Blending in a weaker CNN can only pull a strong XGBoost model's predictions toward the CNN's noisier ones, so for CIP specifically, any weight above 0 is diluting a model that is already working well with one that is not adding useful signal. This is consistent with Section 6: our CIP CNN reproduction already showed the most run-to-run variability of the four antibiotics, so a CNN that is not reliably well-trained is exactly the kind of model you would expect blending to hurt rather than help.

8.4 Why GEN's CNN looks much worse under the paper's fixed threshold than under an optimised one

GEN is the most informative single data point in the whole sweep. At weight 1.00 (pure CNN), the Youden-selected threshold is 0.139 — far below the paper's fixed GEN threshold of 0.505 — and gives recall 0.921 and MCC 0.334. The gap between 0.139 and 0.505 means the CNN's predicted probabilities for GEN resistance are shifted much lower than the fixed threshold assumes. At the fixed threshold, a lot of genuinely resistant isolates get missed simply because the model's raw scores for the resistant class rarely climb above 0.505, even when the model has, in a relative sense, ranked them correctly (which is what the still-reasonable AUC captures).

This matters for how we explain the CNN/GEN gap in Section 6: part of what looks like "the CNN is bad at predicting GEN resistance" may actually be "the CNN's output is miscalibrated for GEN, and the fixed threshold was tuned with CTX/CTZ/CIP's score distributions in mind, not GEN's." We cannot fully

separate a genuinely weaker model from a miscalibrated one with this data, but the size of the threshold gap (0.139 vs. 0.505) is strong evidence that calibration is at least part of the story, not just model quality. This miscalibration also explains why Section 6's ensemble-at-fixed-threshold result for GEN falls further below the paper's than either single model does on its own: the fixed threshold does not get to compensate for a CNN whose scores are shifted low, the way an optimised threshold would.

8.5 A validation approach we had available but did not use

Section 4 mentioned that one of the authors' notebooks, the CNN tuning notebook, does the train/validation/test split correctly: it holds out a validation set from training and never uses the test set for early stopping, unlike every other CNN notebook we reproduced. We did not adapt that notebook's splitting logic into our pipeline — our migration notebook and both sweeps inherited the test-as-validation pattern from the authors' other notebooks instead. This was a missed opportunity: a cleaner version of our reproduction, with a genuine validation set, was sitting in the same repository we were working from. If this project continued past Milestone 3, re-running the CNN and the two sweeps with a proper train/val/test split would be the single highest-value fix, since it would let every number in Sections 6 and 7 be reported without the leakage caveat attached.

9 Deployment

We built a small Streamlit app (`deployment/app.py` in our repository) that serves all four trained CNN/XGBoost pairs. A user picks an antibiotic (CIP, CTX, CTZ, or GEN) and then picks one isolate from a dropdown. That dropdown is not a free-text field: SNP vectors have 60,936 features, which nobody can type by hand, so instead it lists only the 162 isolates in that antibiotic's held-out test split (the same stratified 80/20 split, seed 42, used throughout Sections 6 and 7). Every isolate offered was therefore genuinely unseen by the model being queried.

On clicking "Predict," the app loads that isolate's real SNP row and label from the dataset files, runs it through the antibiotic's CNN and XGBoost models, blends the two probabilities 50/50, and applies the paper's fixed threshold (0.55 for CIP/CTZ, 0.48 for CTX/GEN) to produce a Resistant/Susceptible call. It displays all three probabilities, the prediction, and the isolate's real ground-truth label side by side with a match/mismatch indicator, so the app can demonstrate both correct and incorrect predictions rather than only the former.

We tested this directly rather than assuming it worked: a plain Python script using the same loading and inference functions the app calls confirmed the predicted probabilities are correct for all four antibiotics, and running the app itself in a browser produced both a correct case (CIP, isolate `H108_S5_L001`: CNN 0.084, XGBoost 0.009, ensemble 0.046, predicted and true label both Susceptible) and an incorrect one (GEN, isolate `H105_S3_L001`: CNN 0.504, XGBoost 0.763, ensemble 0.633, predicted Resistant against a true label of Susceptible). Full run instructions, the input/output format, and a screenshot are in `deployment/README.md`.

10 Limitations

Test-set-as-validation leakage. As Sections 4 and 8 discuss at length, the CNN notebooks use the test set for early stopping, the paper's ensemble thresholds appear to have been chosen by checking test performance, and our own weight sweep and k-sweep pick their headline "best" values by the same method. Every

accuracy, MCC, and AUC number in this report should be read as mildly optimistic, not as an unbiased estimate of performance on new data.

Single-seed training. We trained each model once (seed 42), except where Section 6 notes a re-run. CIP's two CNN runs under identical settings produced MCCs of 0.7568 and 0.8405, which tells us run-to-run variance exists but not how large it typically is. None of our point estimates come with a confidence interval.

GEN's apparent CNN weakness is partly a calibration issue, not purely a modelling one. Section 8.4 shows the CNN's own optimal threshold for GEN (0.139) sits far below the paper's fixed threshold (0.505), so some of GEN's poor fixed-threshold performance reflects miscalibrated output scale rather than the model ranking isolates incorrectly. We cannot fully separate these two explanations with the data we have.

No dataset or compute substitution, but no scaling study either. We used the same GieSSen dataset and the same 60,936-feature SNP matrix as the paper, trained on free-tier Colab GPUs. We did not test whether a smaller subset of isolates or features changes these conclusions, beyond the k-sweep's feature-count axis.

Two open items are documented, not fixed. The SNP column duplicate-suffix question (Section 5) is resolved as to *why* duplicates exist, but which gene each of the 16,446 base positions maps to remains unverified against a reference annotation. The k-sweep notebook's SNP-logo plotting code also has a known token-mapping bug (C and G swapped) that does not affect any reported metric, only the logo images, and was left unpatched since fixing it would not change a single number in this report.

The deployment app is a demonstration, not a clinical tool. It only serves predictions for isolates already in our test split; it does not accept an arbitrary new genome, and it carries the same leakage and calibration caveats as every number it was built from.

11 Conclusion

We reproduced AMR-EnsembleNet's 1D CNN and XGBoost models for all four antibiotics, matching the paper's XGBoost results closely (within about 0.02 MCC on every antibiotic) and its CNN results less consistently (CTX above the paper, CTZ close, CIP and GEN below it). At the paper's fixed ensemble thresholds, our ensemble MCCs trail the paper's by 0.02–0.08, with GEN showing the largest gap, which Section 8 traces mainly to threshold miscalibration rather than a weaker model. Our own experiment asked a question the paper never tested: is the paper's fixed 50/50 CNN/XGBoost blend, over all 60,936 features, actually the best choice? The answer is no on both counts for every antibiotic by AUC (the best-AUC weight sits at or near pure XGBoost, 0.00–0.10, for all four), though the best-MCC weight varies more (CTX leans CNN at 0.65, CTZ leans XGBoost at 0.25, GEN is roughly balanced), and a few hundred to a few thousand SNPs, selected by a simple chi-squared test, often match or beat using all 60,936. Finally, the Streamlit app in Section 9 turns this pipeline into something a user can actually query, closing the loop from paper to a working, if modest, deployed demonstration.

12 References

1. Md. Saiful Bari Siddiqui and Nowshin Tarannum. 2026. Fusing Sequence Motifs and Pan-Genomic Features: Antimicrobial Resistance Prediction using an Explainable Lightweight 1D CNN–XGBoost Ensemble. In *SCA/HPCAsia 2026: Supercomputing Asia and International Conference on High Performance Computing in Asia Pacific Region*, January 26–29, 2026, Osaka, Japan. ACM. https://doi.org/10.1145/3773656.3773682 — the paper we reproduced.

2. Deborah J. Eaves, Luke Randall, Douglas T. Gray, Antony Buckley, Martin J. Woodward, Allan P. White, and Laura J. V. Piddock. 2004. Prevalence of Mutations within the Quinolone Resistance-Determining Region of *gyrA*, *gyrB*, *parC*, and *parE* and Association with Antibiotic Resistance in Quinolone-Resistant *Salmonella enterica*. *Antimicrobial Agents and Chemotherapy* 48, 10 (2004), 4012–4015.

3. David C. Hooper. 2001. Mechanisms of action of antimicrobials: focus on fluoroquinolones. *Clinical Infectious Diseases* 32, Supplement_1 (2001), S9–S15.

4. Alexandre Drouin, Sébastien Giguère, Maxime Déraspe, Mario Marchand, and Jacques Corbeil. 2016. Predictive computational phenotyping and biomarker discovery using reference-free genome comparisons. *BMC Genomics* 17, 754 (2016).

5. Sebastian M. Gygli, Peter M. Keller, Marie Ballif, Nicolas Blöchliger, Rico Hömke, Miriam Reinhard, Chloé Loiseau, Erik C. Böttger, and Sebastien Gagneux. 2019. Whole-Genome Sequencing for Drug Resistance Profile Prediction in *Mycobacterium tuberculosis*. *Antimicrobial Agents and Chemotherapy* 63, 4 (2019).

6. Bo Li, Nanxi Zhang, You-Gan Wang, Andrew W. George, Antonio Reverter, and Yutao Li. 2018. Genomic prediction of breeding values using a subset of SNPs identified by three machine learning methods. *Frontiers in Genetics* 9 (2018), 237.

7. Jinhong Shi, Yan Yan, Matthew G. Links, Longhai Li, Jo-Anne R. Dillon, Michael Horsch, and Anthony Kusalik. 2019. Antimicrobial resistance genetic factor identification from whole-genome sequence data using deep feature selection. *BMC Bioinformatics* 20, Suppl 15 (2019), 535.

8. Aikaterini Sakagianni, Christina Koufopoulou, Georgios Feretzakis, Dimitris Kalles, Vassilios S. Verykios, Pavlos Myrianthefs, and Georgios Fildisis. 2023. Using Machine Learning to Predict Antimicrobial Resistance—A Literature Review. *Antibiotics* 12, 3 (2023), 452.

9. Yunxiao Ren, Guoxia Tie, Yang Jin, Shuyu Wang, Renqi Wang, and Hui Wang. 2022. Prediction of antimicrobial resistance based on whole-genome sequencing and machine learning. *Bioinformatics* 38, 2 (2022), 325–334. — original source of the GieSSen *E. coli* SNP matrix and phenotype labels, and of the ML-iAMR codebase we used to resolve the SNP integer encoding (Section 5).

10. Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. 2019. BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics*, Volume 1. ACL, 4171–4186.

11. Yanrong Ji, Zhihan Zhou, Han Liu, and Ramana V. Davuluri. 2021. DNABERT: pre-trained Bidirectional Encoder Representations from Transformers model for DNA-language in genome. *Bioinformatics* 37, 15 (2021), 2112–2120.

12. Shuang Zhang, Rui Fan, Yuti Liu, Shuang Chen, Qiao Liu, and Wanwen Zeng. 2023. Applications of transformer-based language models in bioinformatics: a survey. *Bioinformatics Advances* 3, 1 (2023), vbad001.

13. Sjoerd van den Belt, Hanqing Zhao, and Nikolaos Alachiotis. 2024. Scalable CNN-based classification

of selective sweeps using derived allele frequencies. *Bioinformatics* 40, Supplement_2 (2024), ii29–ii36.

14. Shikha Sharma, Nidhi Kalra, and Rinkle Rani. 2025. Exploring Genomic Sequence Encoding Methods for 1D Convolutional Neural Networks: A Comparative Study. In *Beyond Artificial Intelligence. AICTA 2023*. Lecture Notes in Networks and Systems, Vol. 1326. Springer, Singapore, 651–663.

15. Leo Breiman. 2001. Random Forests. *Machine Learning* 45, 1 (2001), 5–32.

16. Tianqi Chen and Carlos Guestrin. 2016. XGBoost: A Scalable Tree Boosting System. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*. ACM, 785–794.

17. ML-iAMR GitHub repository. https://github.com/YunxiaoRen/ML-iAMR — source of the raw SNP matrix and phenotype files, and the preprocessing script and documentation used to resolve the SNP integer-to-nucleotide encoding and the duplicate-column question.

18. AMR-EnsembleNet GitHub repository (original authors' code). https://github.com/Saiful185/AMR-EnsembleNet — the codebase whose notebooks we reproduced.

19. Our reproduction repository. https://github.com/oliGAMER/ML_Paper_Reprod, branch `M3_Own_Experiment` — our team's code for the reproduction and the two additional experiments described in Sections 6 and 7.